Coinbase SEA: Insights to Impact Challenge

Word count: ~1,490 (AI disclosure included; code blocks excluded). **Assumptions:** fictional data, UTC timestamps, Snowflake SQL. Code is abridged; full code and tests: github.com/armaansingla14/coinbase-insights-to-impact.

Exercise 1: Design the Data Model

Goal: one row per user, so the PM can see which step leaks most for each sign-up cohort, without joins.

Layers

Layer	Grain	Job
`stg_app`, `stg_kyc`, `stg_ledger`	source event	One per source table, same output: UTC timestamps, `lower(trim())` IDs, 5 steps; unioned as `stg_onboarding_events`
`int_identity_map`	source ref → `user_uuid`	Deterministic links only (accounts, anon-id↔login)
`int_onboarding_quarantine`	rejected event	Unmatched refs, unreadable timestamps, with reason
`fct_onboarding_events`	user × step	First occurrence, plus `n_source_events` so retries stay visible
`fct_onboarding_funnel`	user	`*_at` per step, `reached_*` flags, DQ flags

Keys. `user_uuid` from the accounts system of record is the only join key. Emails, devices and anonymous IDs only resolve *to* it. No fuzzy matching: a few extra matches aren't worth an unauditable key. Per step, backend records (KYC decision, ledger deposit, trade fill) outrank client events, which blockers and retries distort.

Timestamps. One parser per format (epoch, ISO with offset, local time plus zone column), all to `event_ts_utc`.

Missing and out-of-order events: flag them, don't "fix" them

- *Missing step* (deposit but no KYC event): `reached_identity_verified = TRUE`, since depositing requires KYC, but `identity_verified_at` stays NULL and `has_missing_step = TRUE`. Conversion stays correct; time-to-step excludes the gap. (If card buys skip deposits, "deposit" must mean any funding event.)
- *Missing sign-up*: fall back to `accounts.created_at`, set `signup_event_missing`. Accounts are the spine, so users with zero events still count.
- *Out of order*: keep real timestamps, set `is_out_of_order` (compared against *every* earlier step, so a gap can't hide it).
- *Duplicates or retries*: earliest per user × step, deterministic tie-break.
- *Late data*: rebuild users with a new event or identity link in 7 days, merging on `user_uuid` so reruns are safe. Segments join as of `signup_at`.

- *Immature cohorts*: compare conversion within a fixed window from sign-up (e.g. 7 days).

```
fct_onboarding_events AS (              -- one row per user x step
  SELECT m.user_uuid, e.step, e.event_ts_utc, e.source, e.source_event_id,
         COUNT(*) OVER (PARTITION BY m.user_uuid, e.step) AS n_source_events
  FROM stg_onboarding_events e
  JOIN int_identity_map m ON m.user_ref = e.user_ref   -- misses -> int_onboarding_quarantine
  WHERE e.event_ts_utc IS NOT NULL                     -- parse failures -> quarantine too
  QUALIFY ROW_NUMBER() OVER (PARTITION BY m.user_uuid, e.step
                             ORDER BY e.event_ts_utc, e.source, e.source_event_id) = 1
)
SELECT COALESCE(p.user_uuid, a.user_uuid)                          AS user_uuid,
  COALESCE(p.signup_at, a.created_at)                              AS signup_at,
  p.signup_at IS NULL                                              AS signup_event_missing,
  (p.first_deposit_at IS NOT NULL OR p.first_trade_at IS NOT NULL) AS reached_first_deposit,
  -- reached_* / has_missing_step: one clause per step (deposit shown); out-of-order: vs ANY step
  (p.first_deposit_at IS NULL AND p.first_trade_at IS NOT NULL)    AS has_missing_step,
  COALESCE(p.is_out_of_order, FALSE)                               AS is_out_of_order
  -- full model: ex1/fct_onboarding_funnel.sql; SQLite port + 32 tests: ex1/sqlite, ex1/tests
FROM pivoted p
FULL OUTER JOIN accounts a ON a.user_uuid = p.user_uuid;  -- spine: zero-event users still count
```

```
-- What the PM runs: 7-day conversion by weekly sign-up cohort (tested in ex1/sqlite)
SELECT DATE_TRUNC('week', signup_at) AS cohort_week, COUNT(*) AS signups,
       AVG(IFF(COALESCE(first_deposit_at, first_trade_at)          -- a trade implies a deposit
               < DATEADD('day', 7, signup_at), 1, 0))     AS deposit_7d,
       AVG(IFF(first_trade_at   < DATEADD('day', 7, signup_at), 1, 0)) AS trade_7d
FROM fct_onboarding_funnel
WHERE signup_at < DATEADD('day', -7, CURRENT_TIMESTAMP())       -- immature cohorts excluded
GROUP BY 1 ORDER BY 1;
```

Data quality checks (*error* blocks the build and pages the owner; *warn* flags trends)

- Grain: `user_uuid` unique, not null; `(user_uuid, step)` unique; accepted `step` values; `user_ref` unique in `int_identity_map`.
- Identity match rate ≥ 99%. Nothing is silently dropped: staged = used + quarantined.
- Reconciliation: funnel sign-ups vs. `accounts` created per day, within 0.5%.
- No future or pre-2012 timestamps (parse failures). Out-of-order < 2%. Freshness < 6h.
- Alert when the `ts_was_assumed_utc` rate jumps (a client release dropped the offset).

Verified in SQLite on 15 messy users plus 400 generated; every check fires when its fault is planted. Testing caught real first-draft bugs (upper-case UUIDs lost deposits), now regression tests.

Exercise 2: The Data Doesn't Add Up

First instinct: check the *grain* before comparing totals, starting with C's duplicate IDs. **Two hours:** 30 min on grain and duplicates, 60 confirming definitions with system owners, 30 on the PM note. To test the script, I planted the usual suspects (replays, timezone windows, filters) in synthetic 52K/45K/57K extracts.

```
# ex2/reconcile_signups.py (stdlib only; abridged. Full script + unit tests in ex2/)
PT = ZoneInfo("America/Los_Angeles")  # handles PST/PDT switches
PARSERS = {
    "iso": parse_iso,
    "epoch_ms": lambda v: datetime.fromtimestamp(float(v) / 1000, tz=UTC),
    "naive_pt": lambda v: (datetime.strptime(v, "%Y-%m-%d %H:%M:%S")
                           .replace(tzinfo=PT).astimezone(UTC)),
}

def load(path, id_col, ts_col, ts_fmt):
    ...
            r["uid"] = (r.get(id_col) or "").strip().lower()
            try:
                r["ts_utc"] = PARSERS[ts_fmt]((r.get(ts_col) or "").strip())
            except (ValueError, OverflowError, OSError):
                r["ts_utc"] = None
            if not r["uid"]:
                bad["blank user_id"] += 1
            elif r["ts_utc"] is None:
                bad["unparseable timestamp"] += 1
    ...

def duplicates(rows):
    by_uid = defaultdict(list)
    for r in rows:
        by_uid[r["uid"]].append(r)
    return {u: sorted(rs, key=lambda r: (r["ts_utc"], r.get("ingest_batch_id", "")))
            for u, rs in by_uid.items() if len(rs) > 1}

def why_not_in_b(r, pt_day):
    if is_test(r):
        return "test/internal account"
    if r["ts_utc"] < pt_day[0]:
        return f"created before {pt_day[0]:%H:%M} UTC (previous Pacific day)"
    ...
    if not (r.get("email_verified_at") or "").strip():
        return "email not verified"
    return "UNEXPLAINED"

# in main(): which columns differ between copies of a duplicate, and which systems hold each ID
differing = Counter(col for rs in dups.values() for col in rs[0]
                    if col not in ("uid", "ts_utc") and len({r[col] for r in rs}) > 1)
membership = Counter("".join(s for s in "ABC" if u in ids[s]) for u in set().union(*ids.values()))
```

Output on the synthetic CSVs (abridged):

```
== 1. Duplicate user_ids, starting with System C (keep earliest row per user) ==
C: 4,000 user_ids duplicated; 5,000 extra rows
   extra rows in *_retry batches: 5,000 of 5,000 across 12 batches
B:  45,000 rows |  45,000 distinct | 09-24 07:00 -> 09-25 06:59 UTC
A raw                                                    52,000
  - created before 07:00 UTC (previous Pacific day)     -12,000
  - email not verified                                   -5,500
  - test/internal account                                -2,000
  + created on next UTC day (still Pacific report day)  +12,500
= bridged to B                                           45,000   (B raw 45,000)
UNEXPLAINED users: 0
Accounts created, UTC day, excl. test/internal, deduped: 50,000
```

What the synthetic run shows, and how I'd confirm each on real data (cheapest first)

1. **C (57K):** duplicates differ only in `ingest_batch_id`; all 5,000 extras are `_retry` / `_retry2` copies of six 13:00–16:00 UTC batches (b26–b31, hence 12 replay batches): replayed ingestion. Deduplicated, C's IDs match A's. I'd confirm with the pipeline owner and request an upstream uniqueness test.
2. **B (45K):** starts at 07:00 UTC, so B uses a Pacific day. Every A-only user is test, unverified, or pre-07:00 UTC. B means "verified sign-ups, Pacific day, excluding test": a *different metric* with the same name.
3. **A (52K):** includes 2,000 internal/QA accounts.
4. With real data: get each owner's written definition, check bot/KYC-reject filters, re-pull next day for late rows. I don't quote a number until UNEXPLAINED is zero.

What I'd send the PM (same day)

> "Use **50,000 new accounts** (created on 9/24 UTC, excluding internal/test accounts). The systems aren't disagreeing about facts; they count different things: C double-counts replayed batches, B uses Pacific time and only counts email-verified users, and A includes test accounts. I've reconciled all three with zero unexplained users. Suggested footnote: '*Sign-ups = accounts created, UTC day, excl. internal.*' Risk: the test-account filter uses email domains, so the number could move by around ±1% once confirmed with Eng. I'll have that by Thursday."

If earlier decks used B, I'd restate last week on the new basis so the change doesn't read as ~11% growth.

What I flag: C's replayed batches (owner ticket, uniqueness test), B's misleading name (verified, Pacific day), A's test accounts.

Exercise 3: The Inherited Pipeline

What I flag, ordered by impact on the number

#	Issue	Why it matters
1	Joining `devices` **fans out rows**	Each trade repeats per active device, inflating `completed_trades` (2 trades × 3 devices = 6). `COUNT(DISTINCT)` shields `active_traders` within a country, not across countries (see 6). **Same bug as Exercise 4's 41%.**
2	`d.is_active = 1`	*Current* state, not state that week. Web-only users and later-deactivated devices silently vanish; re-running a past week changes it.
3	`status = 1 OR status = 3`	Magic numbers. `active_traders` counts *any* status (even failed), `completed_trades` filters: two definitions.
4	Implicit comma joins	One dropped predicate silently becomes a cross join.
5	Hardcoded dates; timezone unknown	Not really "weekly"; boundaries may shift by country.
6	`u.country`	Current or sign-up? Country history fans out too: user counted in both countries, trades doubled.

The snippet already groups by country, so the cut is likely lost in a later rollup.

How I'd get the country cut out without breaking the metric

1. **Freeze a baseline.** Snapshot 12 weeks so any change is measurable, not arguable.
2. **Map the blast radius.** Lineage shows downstream dashboards and owners.
3. **Characterization tests.** Pin current totals; my change must reproduce them.
4. **Additive delivery.** Expose country with existing logic *unchanged*, reconciled to the baseline, labeled "*Legacy definition; known issues in DATA-123.*" The PM gets the cut before Friday; nobody's number moves silently.
5. **Fix in the open.** Build `v2` side by side, quantify the difference, switch only after metric-owner sign-off.

To the PM: "Country cut by Thursday, matching last week's total. Present trader counts, not trade counts, which multi-device users inflate (the 41% issue); web-only traders are missing. Fixes follow the review."

Verified on a 7-user SQLite fixture: legacy reports 11 completed trades against 8 real; its trader total matches only because dropped web-only traders offset counted failed-only ones.

```
-- v2 sketch: dedupe to user-week before counting; statuses from a lookup, not literals
WITH trades_wk AS (                                  -- 1 row per user-week (tested)
  SELECT t.user_id,
         -- assumes created_at is TIMESTAMP_TZ (if NTZ: 3-arg CONVERT_TIMEZONE); WEEK_START = 1
         DATE_TRUNC('week', CONVERT_TIMEZONE('UTC', t.created_at)) AS week,
         COUNT_IF(s.is_completed)                                  AS completed_trades
  FROM trades t
  LEFT JOIN dim_trade_status s ON s.status_id = t.status   -- unmapped status => DQ test fails
  GROUP BY 1, 2
  -- DEFINITION CHANGE: active trader = >=1 completed trade; needs owner sign-off
  HAVING COUNT_IF(s.is_completed) > 0
)
SELECT w.week, COALESCE(u.signup_country, 'UNKNOWN') AS country,
       COUNT(*) AS active_traders, SUM(w.completed_trades) AS completed_trades
FROM trades_wk w
LEFT JOIN dim_user u ON u.user_id = w.user_id    -- sign-up country (owner to confirm); 1 row/user
GROUP BY 1, 2;
```

Using AI. To explain the 400 lines, map CTE dependencies, and draft tests. Its output is a hypothesis until verified:

- Diff old vs new at every grain (total, week, country), attributing each difference to a change.
- Hand-trace edge-case users (single/multi-device, web-only, failed-only, NULL country) through both versions, as pinned tests.
- Confirm status codes with the source enum or owning team; an AI's guess about `3` isn't documentation.

Exercise 4: Your Number Is "Wrong"

First, what are we counting? The 41% divides trading user-*device* pairs by weekly active *users*, so it isn't a share of users. I first reproduce both from my model: user-device pairs give 41%, users give 34%. The gap

is Exercise 3's fan-out (a 100-user fixture confirms: 7 two-device traders turn 34% into 41%).

What I'd say to the PM, privately and right away:

> "I won't derail your review. But forcing my table to 41% gives us two broken tables instead of one. 41% counts trading devices; 34% counts people. Trading didn't drop; only the counting changed. Better we raise the double count than leadership finds it. Here's how we keep Friday clean."

For Friday (neither starts a live debate)

- **Option A:** Present 41% footnoted: "*Counts devices, not users; definition under review.*"
- **Option B:** The PM adds one line: "*We found a device double-count; the user-level figure is ~34%, with a full reconciliation next review.*"

My recommendation: Option B. Option A only if no decision in the deck depends on the level. Either way, the PM presents it; I don't correct them in the room.

After the review: moving to one trusted definition

1. **Quantify the gap** by week and segment. Key question: *does it change any decision, or only the level?* The gap tracks multi-device adoption; if that shifted, past trend calls were biased too.
2. **Audit decisions.** Flag goals or experiments that crossed a threshold only due to inflation.
3. **Write one definition** in the semantic layer (grain, filters, owner). Growth leadership signs off, not me.
4. **Run both in parallel**, clearly labeled, for about 2 review cycles, so the gap is understood before the switch.
5. **Cut over with a changelog.** Restate history, retire the legacy tile, and add a test that fails if the grain exceeds one row per user.

AI Disclosure

I used Claude Code (Anthropic), running several agents in parallel, to draft these written responses, generate the synthetic test data, and write and test the Python and SQL. I directed the work, set the requirements, and reviewed the final submission. All numbers and output shown come from running that code.